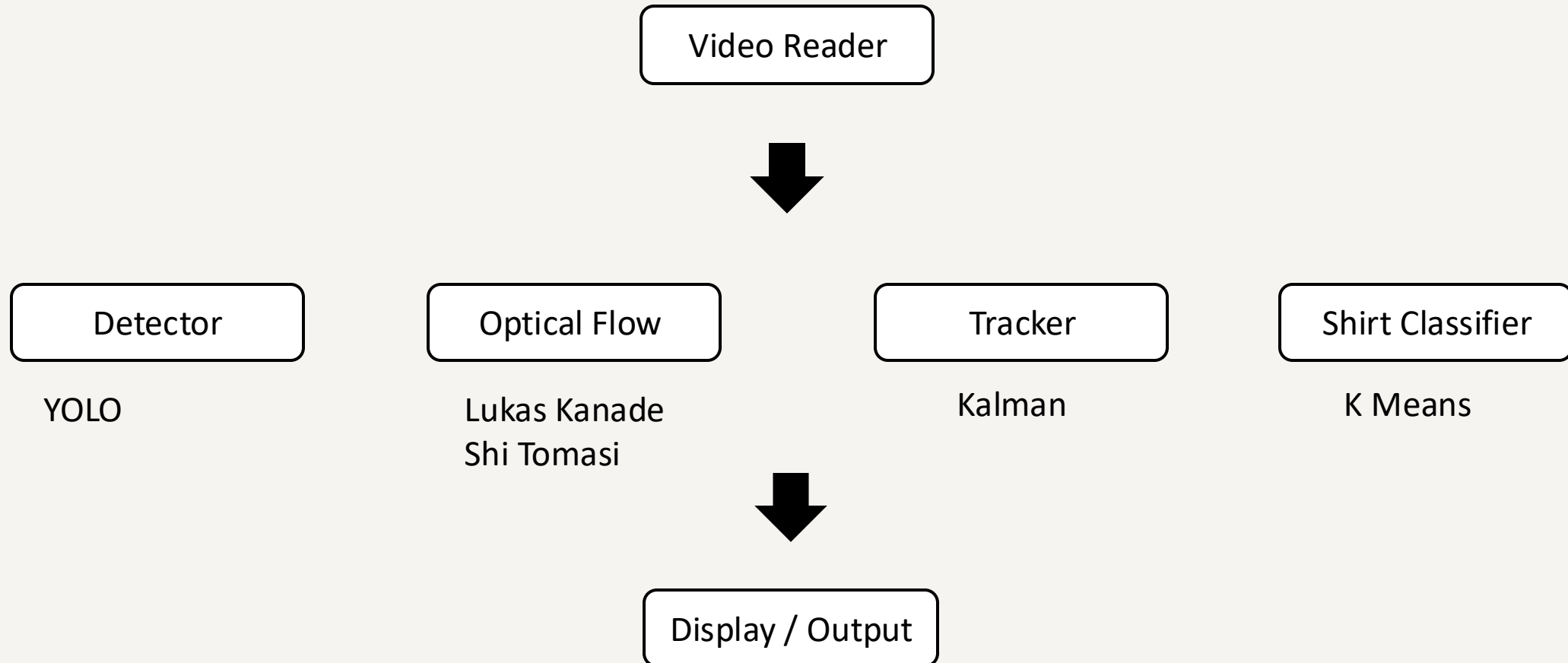




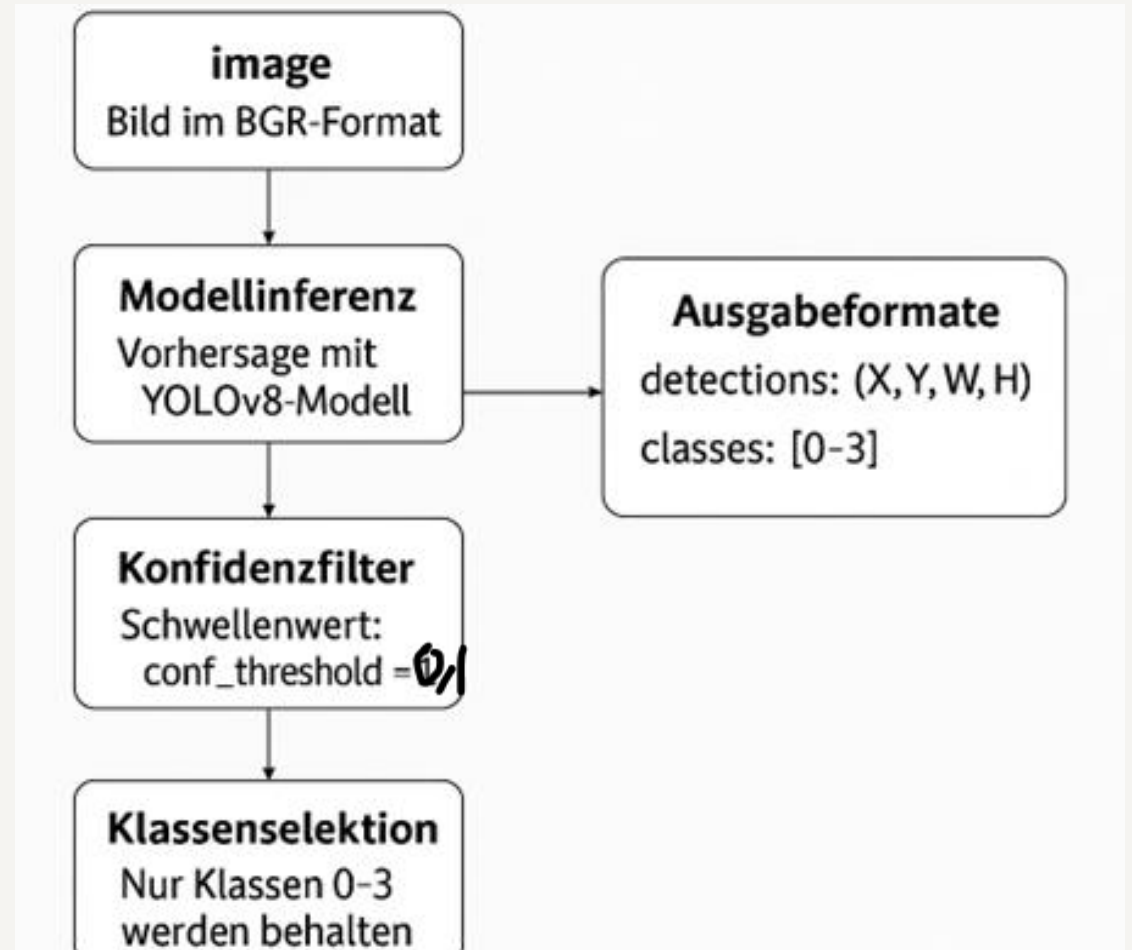
# Soccer Tracking Projekt

# Code Struktur – General



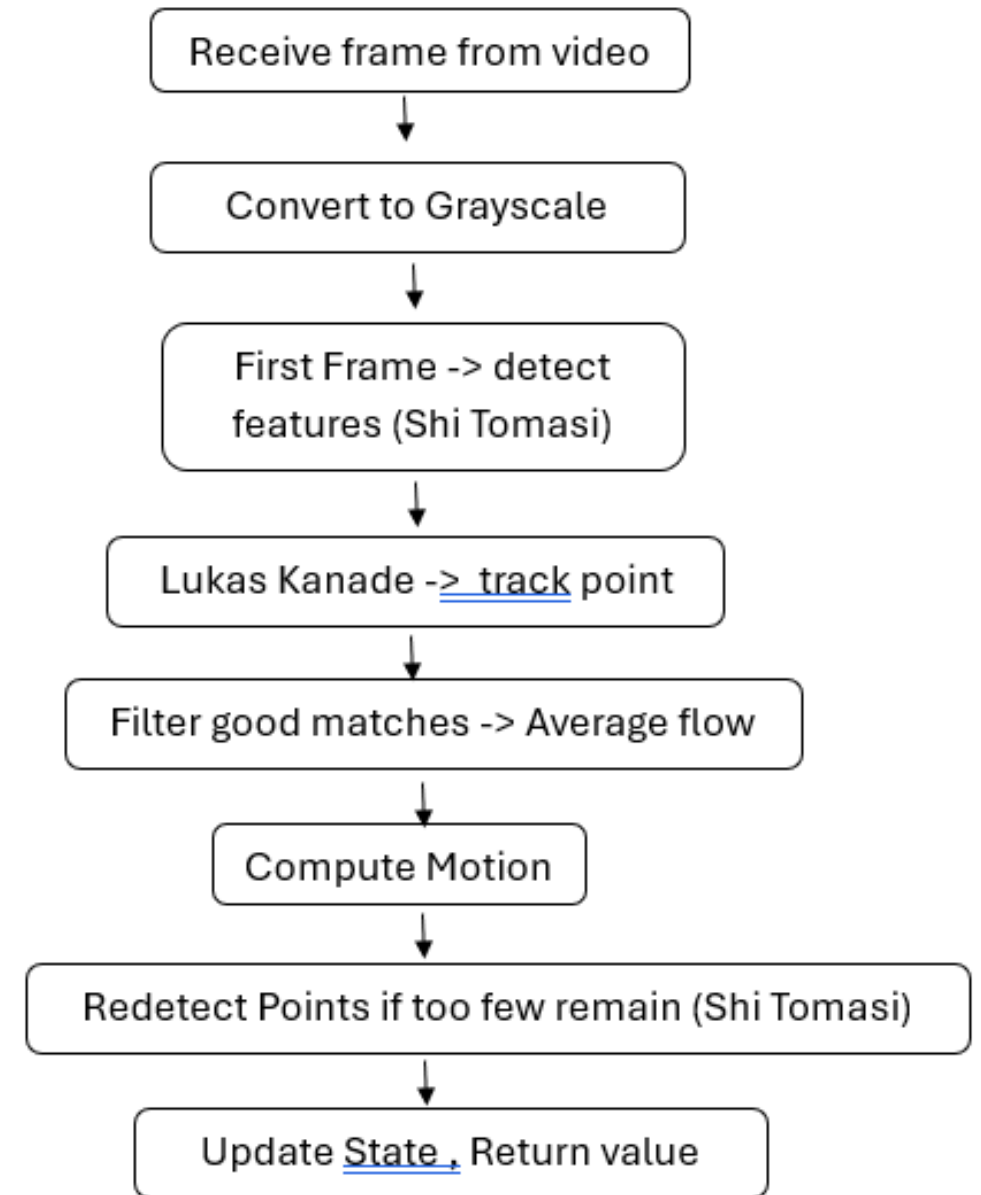
# Detector

- Erkennung von Ball, Torhüter, Spieler und Schiri mithilfe eines vortrainierten YOLOv8-Modells.
- Warum YOLO?
- Schwellenwert für Konfidenz:  
*conf\_threshold = 0.1*
- Ausgabeformate:  
Bounding Boxes: (X, Y, W, H)  
Klassen: [0–3]



# Optical Flow

- Merkmalsdetektion: Shi-Tomasi
- Bewegungsverfolgung: Lukas-Kanade
- Anwendung: Spieler-Tracking im Video
- `cv2.goodFeaturesToTrack` -> Shi-Tomasi  
Findet gute Ecken zum Verfolgen (z. B. Trikots, Gliedmaßen)
- `cv2.calcOpticalFlowPyrLK` -> Lukas Kanade  
Verfolgt diese Punkte Bild für Bild mit lokalem Patch-Matching.
- Rot: Durchschnittsbewegung (Teamrichtung)



# Optical Flow

- Warum Lukas Kanade + Shi Tomasi ?

|                               |                                                        |
|-------------------------------|--------------------------------------------------------|
| Effizient für Echtzeit        | Leichtgewichtig, CPU-Freundlich                        |
| Robust bei kleinen Bewegungen | Geeignet für die Verfolgung langsam bewegender Spieler |
| Kein Modelltraining nötig     | Reine klassische Methode                               |
| Automatische Wiedererkennung  | Unterstützt die Neuererkennung von Merkmalen           |

# Tracker

- Filter : Ein einzelnes Objekt mit Kalman-Filter
- Tracker : Verwaltung aller Filter (also aller Objekte)

## Kalman Filter , Euclidean Distance

- **self.x**: Zustandsvektor [x, y, w, h, vx, vy] → Position, Größe, Geschwindigkeit.
- **self.P**: Anfangsunsicherheit (besonders hoch für Geschwindigkeit).
- **self.F**: Zustandsübergangsmatrix (Modell: konstante Geschwindigkeit).
- **self.H**: Beobachtungsmatrix → nur Position & Größe beobachtbar.
- **self.id**: Eindeutige ID für jedes Objekt.
- **self.age**: Alter des Objekts in Frames.
- **self.misses**: Wie viele Frames das Objekt nicht erkannt wurde.

## Predict ()

**self.x = self.F @ self.x** -> berechnet den vorhergesagten Zustand mithilfe der Zustandsübergangsmatrix (F)

**self.P = self.F @ self.P @ self.F.T + self.Q** -> Hier wird die Kovarianzmatrix P aktualisiert.

**self.age += 1** -> Erhöht das Alter des Trackings

**self.misses += 1** -> Zählt, wie oft das Objekt nicht erkannt wurde

# Tracker

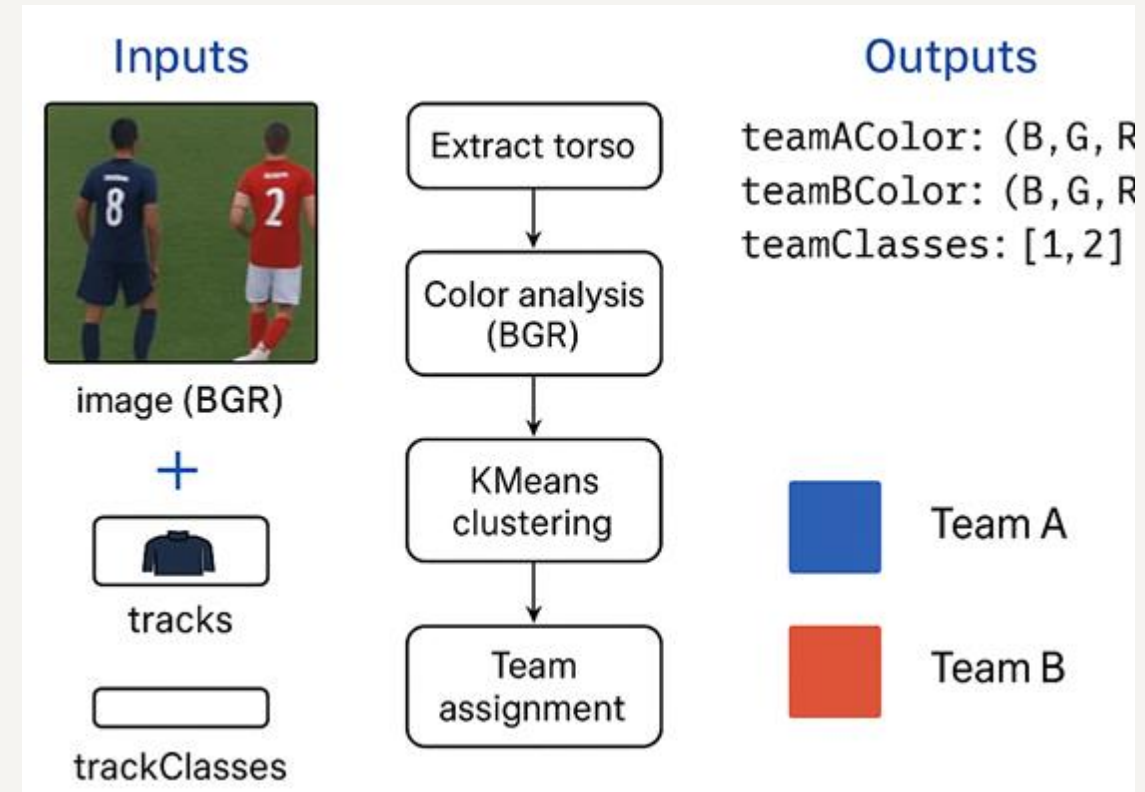
"tracks"  
"trackVelocities"  
"trackAge"  
"trackClasses"  
"trackIds"

## Ablauf der step() Funktion

| Phase                 | Beschreibung                                                                                                          | Code                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1. Prediction         | Jeder Track sagt ihre nächste Position voraus<br>( <a href="#">Kalman predict</a> )                                   | <a href="#">f.predict()</a>              |
| 2. Matching           | Neue Detektionen werden mit bestehenden Tracks<br>per <a href="#">euklidischer Distanz</a> verglichen                 | <a href="#">np.linalg.norm(...)</a>      |
| 3. Update             | Falls eine passende Detektion gefunden wird -><br>update des Filters                                                  | <a href="#">f.update(dets[best_idx])</a> |
| 4. Neuer Track        | Unzugeordnete Detektionen -> Neuer Track (neue<br>Filter)                                                             | <a href="#">self.filters.append(nf)</a>  |
| 5. Remove (old track) | Track ohne Updates: Zähler misses erhöht -> Wenn<br><a href="#">misses &gt;= max_misses</a> , wird der Track gelöscht | <a href="#">is_deleted()</a>             |

# Shirt Classifier

- Für jeden Spieler wird der obere Bereich der Bounding Box ausgeschnitten.
- Aus dem Torso wird der durchschnittliche Farbwert im BGR-Format berechnet.
- **KMeans-Clustering:**
  - Clustert alle Spielerfarben in 2 Gruppen.
  - Der Cluster mit dem niedrigeren Mittelwert wird als Team A gesetzt.
- Teamzuordnung: Jeder Spieler wird seinem Cluster entsprechend als Team A (1) oder Team B (2) klassifiziert.





# Vergleich der Trikot-Farbklassifikatoren

|                                         |  KMeans |  | His  gram |
|-----------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Keine Farb-Vorannahmen                  | ✓                                                                                         | ✗                                                                                   | ✗                                                                                            |
| Flexibel bei wechselnde Farben          | ✓                                                                                         | ✗                                                                                   | <del>✓</del>                                                                                 |
| Reproduzierbare Gruppierung             | ✗                                                                                         | ✓                                                                                   | ✓                                                                                            |
| Beste Option für bekannte Trikot-farben | ✗                                                                                         | ✓                                                                                   | ✓                                                                                            |

# Herausforderungen

|                                                                |                                                                                                        |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Kleine Objekte schwer zu erkennen                              | Ähnliche Trikotfarben<br>KMeans und Durchschnittsfarbe vom Torso                                       |
| Lichtverhältnisse & Schatten<br>Farbmittelwert wurde berechnet | Bewegungsschärfe & Überdeckungen<br>Tracker erlaubt fehlende Frames und nutzt predict() zur Vorhersage |

**Vielen Dank**

## Optical Flow - Parameters

```
# Shi-Tomasi corner detection parameters
self.feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
```

```
# Lucas-Kanade optical flow parameters
self.lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)
```

## Tracker - Parameters

```
# Initialize state vector with 6 elements: [x, y, w, h, vx, vy]
self.x = np.zeros((6,), dtype=np.float32)
self.x[:4] = z # set initial position and size based on detection
self.cls = int(cls) # object class
```

```
# Observation matrix (H): only measures position and size (4D)
self.H = np.zeros((4, 6), dtype=np.float32)
self.H[0, 0] = self.H[1, 1] = self.H[2, 2] = self.H[3, 3] = 1.0
```

```
# Initial state covariance matrix (P): higher uncertainty in velocity
self.P = np.diag([10, 10, 10, 10, 1000, 1000]).astype(np.float32)
```

```
# State transition matrix (F): assumes constant velocity model
self.F = np.eye(6, dtype=np.float32)
self.F[0, 4] = 1.0 # x += vx
self.F[1, 5] = 1.0 # y += vy
```

## Tracker - Parameters

```
def predict(self):
    self.x = self.F @ self.x # predict next state
    self.P = self.F @ self.P @ self.F.T + self.Q # update covariance
    self.age += 1
    self.misses += 1 # increase missed count
```

```
def update(self, z):
    z = np.asarray(z, dtype=np.float32)
    y = z - self.H @ self.x # innovation
    S = self.H @ self.P @ self.H.T + self.R # innovation covariance
    K = self.P @ self.H.T @ np.linalg.inv(S) # Kalman gain
    self.x = self.x + K @ y # update state
    I = np.eye(6, dtype=np.float32)
    self.P = (I - K @ self.H) @ self.P # update covariance
    self.misses = 0 # reset misses on successful update
```

```
def __init__(self):
    self.name = "Tracker"
    self.filters = [] # list of active tracks
    self.max_misses = 5 # allowed misses before deleting a track
    self.dist_thresh = 50.0 # max Euclidean distance for matching
    self.frame_count = 0
```

```
# Matching detections to existing filters based on Euclidean distance
for f in self.filters:
    best_idx = -1
    best_dist = float("inf")
    for i in unmatched_dets:
        dist = np.linalg.norm(f.get_state()[2:] - dets[i][2:])
        if dist < best_dist and dist < self.dist_thresh:
            best_idx = i
            best_dist = dist
    if best_idx >= 0:
        f.update(dets[best_idx])
        f.cls = classes[best_idx]
        unmatched_dets.remove(best_idx)
```